PERSONAL PORTFOLIO

Backend Proje Planı

FastAPI · SQLModel · Alembic · PostgreSQL · Docker

Mevcut yapı denetimi ve 14 günlük uygulama planı

18 Ağustos – 31 Ağustos 2026

Hazırlanma tarihi: 18 Ağustos 2026

İçindekiler

1. Mevcut Durum Değerlendirmesi
- 1.1 İyi durumda olanlar
- 1.2 Açık sorunlar

2. Kapsam, Hedef ve Başarı Kriterleri
- 2.1 Amaç
- 2.2 Kapsam — Dahil
- 2.3 Kapsam — Hariç (bilinçli olarak dışarıda)
- 2.4 Başarı Kriterleri (ölçülebilir)

3. Hedef Mimari

4. Faz ve Görev Dökümü
- Faz 0 — Temizlik ve Zemin · Gün 1 (18 Ağu, Sal) · ~4,5 sa
- Faz 1 — Router, Şema ve Repository Hijyeni · Gün 2–3 (19–20 Ağu) · ~7,5 sa
- Faz 2 — Test Altyapısı · Gün 3 (20 Ağu) · ~4 sa
- Faz 3 — Alembic ve PostgreSQL Geçişi · Gün 4 (21 Ağu, Cum) · ~5 sa
- Faz 4 — Blog (Post) Modülü · Gün 5–6 (22–23 Ağu, hafta sonu) · ~8 sa
- Faz 5 — Admin Auth (JWT) · Gün 7–8 (24–25 Ağu) · ~8 sa
- Faz 6 — Medya (ImageKit) ve İletişim Formu · Gün 9–10 (26–27 Ağu) · ~8,5 sa
- Faz 7 — Docker ve Droplet Deployment · Gün 11–13 (28–30 Ağu) · ~10,5 sa
- Faz 8 — Kapanış ve Devreye Alma · Gün 14 (31 Ağu, Pzt) · ~4 sa

5. Zaman Çizelgesi

6. Riskler ve Önlemler

7. İlk Adım

Kaynaklar

1. Mevcut Durum Değerlendirmesi

Kodun tamamı okundu ve uygulama izole bir ortamda gerçekten çalıştırılarak test edildi (bağımlılıklar kuruldu, `TestClient` ile endpoint'lere istek atıldı). Aşağıdaki bulgular varsayım değil, **doğrulanmış** sonuçlardır.

1.1 İyi durumda olanlar

Konu	Değerlendirme
Uygulama ayağa kalkıyor	`GET /` → 200, `GET /api/projects/` → 200, `POST /api/projects/` → 200, `/openapi.json` → 200. Depodaki `bug_report.json` (3 Ağustos) 6 blocker listeliyordu; **hepsi kapanmış**: import root'ları düzeltilmiş, `ARRAY` → `JSON` yapılmış, eksik `await`'ler eklenmiş. Ciddi bir ilerleme var.
Katman ayrımı	`api/` → `repositories/` → `models/` + `schemas/` ayrımı doğru kurgulanmış. Bu, çoğu portfolyo projesinde olmayan bir olgunluk.
Async tutarlılığı	`create_async_engine` + `async_sessionmaker(class_=AsyncSession)` + `AsyncGenerator` dependency zinciri doğru kurulmuş.
Güncel FastAPI kalıpları	`lifespan` context manager kullanılmış (deprecated `@app.on_event` değil). Doğru tercih.
Konfigürasyon	`pydantic-settings` + `BaseSettings` + `.env` — hardcoded değer yok.
Generic repository	`BaseRepository[ModelType]` ile CRUD tekrarı önlenmiş. Doğru fikir, sadece sınırları netleşmeli (bkz. B-09).

1.2 Açık sorunlar

Öncelik sırasına göre. Her biri kod okunarak veya çalıştırılarak doğrulandı.

Blocker

B-01 — Yazma endpoint'leri tamamen korumasız

`POST /api/projects/` herkese açık. Yayına çıktığın anda internetteki herkes portfolyona proje ekleyebilir. `core/security.py` tamamen yorum satırından ibaret; üstelik yorumdaki `verify_password` düz metin karşılaştırması yapıyor (`plain_password == hashed_password`) ve dosyada `print(cryptography.__builtins__)` debug satırı duruyor. Auth, "sonra eklerim" kalemi değil — Faz 5'te kapatılıyor.

B-02 — requirements.txt kurulamıyor

`aiosqltite` yazım hatası var (doğrusu `aiosqlite`) → `pip install -r requirements.txt` doğrudan hata veriyor. Ayrıca kodda kullanılan `pydantic-settings` ve async SQLAlchemy'nin ihtiyaç duyduğu `greenlet` listede yok; `databases` paketi listede ama hiç kullanılmıyor. Hiçbir sürüm pinlenmemiş. Yani proje şu an **temiz bir makinede kurulamaz** — Docker'a geçerken bu ilk duvara çarpacağın nokta.

B-03 — myvenv/ klasörü backend içinde ve .gitignore kapsamı dışında

`.gitignore` yalnızca `.env` ve iki `__pycache__` yolunu içeriyor. Dolayısıyla `myvenv/` (binlerce dosya, platforma özel binary'ler), `dev.db` ve `v1_db.db` git tarafından takip ediliyor olma ihtimali yüksek. Depoyu şişirir, başka makinede çakışır, ve veritabanı dosyaları repoya sızar.

Yüksek

B-04 — POST body'de id zorunlu; primary key'i client belirliyor

Doğrulandı: `id` göndermeden POST → **422 Field required**. `id: 99` göndererek POST → **200**, kayıt 99 id'siyle oluşuyor. Sebep: `create_project` request body olarak `ProjectBase` alıyor ve o şemada `id: int` zorunlu. Var olan `ProjectCreate` şeması hiç kullanılmıyor — kullanılsaydı da patlardı, çünkü içinde `slug` yok ama DB'de zorunlu.

B-05 — ORM tablo modeli response_model olarak dönülüyor

`response_model=list[Project]` ve `response_model=Project` — yani SQLModel table model'i doğrudan API şeması. Bugün zararsız görünüyor ama Faz 5'te `User` modeli geldiğinde `hashed_password` gibi alanların API'ye sızması bu kalıpla bir satırlık hata meselesi. Read şeması ayrı olmalı.

B-06 — Migration yok, create_all var

`init_db()` her açılışta `SQLModel.metadata.create_all` çağırıyor. `create_all` **yalnızca olmayan tabloyu yaratır; var olan tabloya kolon eklemez**. `Project`'e yeni bir alan eklediğinde hiçbir şey olmaz, sonra "no such column" hatası alırsın. Postgres'e geçiş planlıyorsan Alembic zorunlu.

B-07 — UPDATE ve DELETE endpoint'i yok

`BaseRepository.update` ve `.delete` yazılmış ama hiçbir route bunları kullanmıyor. Ayrıca `update`, `obj_in.model_dump()` ile **tüm** alanları yazıyor → partial update (PATCH) imkânsız; gönderilmeyen alanlar ezilir.

B-08 — db_dependency'de hata yönetimi yok

Exception durumunda `rollback` yapılmıyor. Yarım kalmış bir transaction ile devam edilebilir.

Orta

B-09 — BaseRepository.get_by_slug base'e ait değil

`self.model.slug` diyor ama `slug` alanı `BaseModel`'de değil, sadece `Project`'te. Slug'ı olmayan bir model (ör. ileride `ContactMessage`) ile çağrılırsa `AttributeError`. Base repository sadece tüm modellerde ortak olan işleri bilmeli.

B-10 — BaseModel her modele name alanını zorunlu kılıyor

Bu bir domain kararı değil, kolaylık kararı. `ContactMessage` veya `Tag` için `name` anlamsız. Base yalnızca `id`, `created_at`, `updated_at` tutmalı.

B-11 — updated_at alanı yok

Blog yazılarında "son güncelleme" göstermek isteyeceksin; sonradan eklemek migration gerektirir.

B-12 — liveDemo modelde var, ProjectBase şemasında yok

Model alanı API'den doldurulamıyor — sessiz bir tutarsızlık. Şema/model senkronizasyonunun test edilmediğinin göstergesi.

B-13 — Boş stub dosyalar ve kullanılmayan router aggregation

`dependencies.py`, `exceptions.py`, `middleware.py`, `api/router.py`, `api/v1/router.py` 0 byte; `core/base.py` yarım import'lardan ibaret; `core/logging.py` tek satır `import logging`. Router aggregation kullanılmadığı için `main.py` her router'ı tek tek import ediyor — 5 modül olunca dağılır.

B-14 — Test yok

`tests/` klasörü mevcut değil. 2 hafta içinde 4 yeni özellik ekleyeceksen regresyon kaçınılmaz. Test altyapısı Faz 2'de, özelliklerden **önce** kuruluyor.

B-15 — CORS origin'leri hardcoded localhost

`["http://localhost:5173", "http://localhost:3000"]` sabit. Droplet'e çıktığında frontend domain'i çalışmaz. `Settings`'ten gelmeli.

B-16 — Python'da camelCase alan adları

`techStack`, `githubLink`, `liveDemo` — PEP 8'e aykırı. Doğru çözüm frontend'i bozmadan: Python tarafında `snake_case`, Pydantic'te `alias_generator=to_camel` + `populate_by_name=True` ile JSON'da camelCase.

Düşük

B-17 — `README.md` UTF-16 kodlanmış; ağaç karakterleri bozuk (`%%%`) görünüyor ve hâlâ `users`/`orders` örneğini anlatıyor, gerçek yapıyı değil.

B-18 — `.env.example` yok; projeyi başka makinede kurmak için hangi değişkenlerin gerektiği belirsiz.

B-19 — `app/api/__init__.py` içinde `from .v1 import *` — wildcard import, gereksiz ve isim çakışması riski.

B-20 — `models/readme.md` ve `schemas/readme.md` var olmayan `blog.py`'yi anlatıyor.

B-21 — `get_all` limiti 1000 sabit, sayfalama metadata'sı (total/page) yok.

B-22 — Yerel Python 3.14 (`.pyc` dosyalarından anlaşılıyor). Çok yeni; bazı paketlerde wheel/uyumluluk sürprizi çıkabilir. Docker imajında 3.12 gibi oturmuş bir sürüm tercih edilmeli.

2. Kapsam, Hedef ve Başarı Kriterleri

2.1 Amaç

Frontend'in (`personal-blog/frontend`) tükettiği, kendi kendine yetebilen bir portfolyo & blog API'si: proje vitrini, blog yazıları, sadece senin erişebildiğin içerik yönetimi, görsel yükleme ve iletişim formu — bir DigitalOcean droplet'inde Docker ile HTTPS üzerinden yayında.

2.2 Kapsam — Dahil

- Project CRUD (public read, admin write)
- Blog Post CRUD (draft/published durumu, slug, markdown içerik, tag)
- JWT tabanlı tek-admin auth; tüm yazma işlemleri korumalı
- ImageKit ile görsel yükleme
- İletişim formu: mesajı kaydet + e-posta bildirimi + rate limit
- Alembic migration altyapısı ve PostgreSQL'e geçiş
- Pytest ile otomatik test paketi
- Multi-stage Dockerfile + docker-compose (api + postgres + reverse proxy)
- Droplet'e deployment, domain, HTTPS, günlük DB yedeği
- GitHub Actions ile test + lint

2.3 Kapsam — Hariç (bilinçli olarak dışarıda)

Bunlar kötü fikir olduğu için değil, **14 güne sığmadığı** için dışarıda. Sonraki iterasyonun malzemesi:

- Yorum sistemi, beğeni, görüntülenme sayacı
- Full-text search (Postgres FTS)
- Çoklu kullanıcı / rol yönetimi (tek admin yeterli)
- Refresh token, sosyal login (OAuth)
- Redis cache, Celery / background worker
- Çok dilli içerik (i18n)
- Admin panel arayüzü (frontend işi)
- Kubernetes, autoscaling, CDN önbellek stratejisi

2.4 Başarı Kriterleri (ölçülebilir)

Proje şu 8 madde sağlandığında "bitti" sayılır:

1. Temiz bir klonda `docker compose up` ile API + Postgres sorunsuz ayağa kalkıyor.
2. `pytest` yeşil; en az 20 test; projects, posts ve auth kapsanıyor.
3. `/api/v1/projects` ve `/api/v1/posts` için tam CRUD; okuma public, yazma admin-only.
4. Token'sız yazma denemesi **401** dönüyor (testle doğrulanmış).
5. Şema sıfırdan `alembic upgrade head` ile kuruluyor; kodda `create_all` çağrısı kalmadı.
6. API droplet'te HTTPS ile canlı ve frontend gerçek API'den veri çekiyor.
7. Depoda `.env`, `*.db` ve `myvenv/` takip edilmiyor.
8. `GET /health` 200 dönüyor (DB bağlantısı dahil kontrol ediliyor).

3. Hedef Mimari

```
backend/
├── app/
│   ├── main.py                # app factory + lifespan + exception handlers
│   ├── config.py              # Settings: ENV, DATABASE_URL, JWT_*, IMAGEKIT_*, SMTP_*, CORS_ORIGINS
│   ├── db.py                  # engine, session maker, get_session (rollback'li)
│   ├── dependencies.py        # get_current_admin, pagination params
│   ├── exceptions.py          # AppError + handler'lar
│   ├── middleware.py          # request-id, timing
│   ├── api/
│   │   ├── router.py          # api_router → v1'i include eder
│   │   └── v1/
│   │       ├── router.py      # v1 aggregation
│   │       ├── projects.py  posts.py  auth.py  media.py  contact.py
│   ├── models/                # base.py project.py post.py user.py contact_message.py
│   ├── schemas/               # base.py common.py(Page[T]) project.py post.py auth.py contact.py
│   ├── repositories/          # base.py project.py post.py user.py
│   ├── services/              # auth_service.py media_service.py email_service.py
│   └── core/                  # security.py (pwdlib+PyJWT), logging.py
├── alembic/versions/
├── tests/                     # conftest.py test_projects.py test_posts.py test_auth.py
├── scripts/create_admin.py
├── alembic.ini · Dockerfile · docker-compose.yml · .env.example · .gitignore
└── requirements.txt + requirements-dev.txt   (veya pyproject.toml + uv)
```

Not: `services/` katmanı yalnızca gerçek iş mantığı olan yerlerde (auth, medya, e-posta) var. Basit CRUD için repository → router doğrudan yeterli; her şeye service yazmak gereksiz katman şişkinliği yaratır.

4. Faz ve Görev Dökümü

Efor tahminleri saat cinsindendir. Bağımlılık belirtilmişse o görev bitmeden başlanmamalı.

Faz 0 — Temizlik ve Zemin · Gün 1 (18 Ağu, Sal) · ~4,5 sa

Yeni özelliğe geçmeden önce projenin **kurulabilir** hale gelmesi gerekiyor. Bu faz kısa ama sonraki her şeyi kilitler.

#	Görev	Efor	Kapattığı
0.1	`.gitignore`'u genişlet: `myvenv/`, `.venv/`, `*.db`, `__pycache__/`, `.env`, `.idea/`, `.vscode/`, `.pytest_cache/`, `.ruff_cache/`	20 dk	B-03
0.2	`git rm -r --cached myvenv dev.db v1_db.db` ile takipten çıkar, commit'le	20 dk	B-03
0.3	`requirements.txt`'i düzelt: `aiosqltite`→`aiosqlite`, `pydantic-settings` + `greenlet` ekle, `databases`'i çıkar, sürümleri pinle. Dev bağımlılıkları `requirements-dev.txt`'e ayır	45 dk	B-02
0.4	`Settings`'i genişlet: `ENV`, `CORS_ORIGINS: list[str]`, `JWT_SECRET`, `JWT_ALGORITHM`, `ACCESS_TOKEN_EXPIRE_MINUTES`	45 dk	B-15
0.5	`.env.example` yaz (gerçek değer yok, sadece anahtarlar)	20 dk	B-18
0.6	`core/security.py`'yi temizle: debug `print` ve ölü yorum bloğunu sil	10 dk	B-01
0.7	`README.md`'yi UTF-8 olarak yeniden yaz; gerçek yapıyı, kurulum ve çalıştırma adımlarını anlat. `models/readme.md` ve `schemas/readme.md`'yi güncelle	45 dk	B-17, B-20
0.8	`bug_report.json`'ı `docs/` altına arşivle veya sil (blocker'ları kapandı)	10 dk	—

Kontrol noktası: Boş bir klasöre `git clone` → `pip install -r requirements.txt` → `uvicorn app.main:app` sorunsuz çalışıyor.

Faz 1 — Router, Şema ve Repository Hijyeni · Gün 2–3 (19–20 Ağu) · ~7,5 sa

Yeni model eklemeden önce mevcut Project modülünü doğru hale getir. Buradaki kalıplar Post, User ve Contact için şablon olacak — yanlış kalıbı 4 kez tekrarlamaktansa bir kez düzelt.

#	Görev	Efor	Bağımlılık	Kapattığı
1.1	`api/v1/router.py` ve `api/router.py`'yi doldur; `main.py` tek bir `include_router(api_router, prefix="/api/v1")` yapsın. Wildcard import'u kaldır	45 dk	Faz 0	B-13, B-19
1.2	Şemaları ayır: `ProjectCreate` (id yok, slug var), `ProjectUpdate` (tüm alanlar `Optional`), `ProjectRead` (id, created_at, updated_at dahil). `ProjectBase`'i request body olmaktan çıkar	1,5 sa	1.1	B-04, B-12
1.3	Tüm `response_model`'leri `ProjectRead` yap; table model'i API yüzeyinden çek	30 dk	1.2	B-05
1.4	Alan adlarını `snake_case`'e çevir (`tech_stack`, `github_link`, `live_demo`) + Pydantic `alias_generator=to_camel, populate_by_name=True` ile JSON'da camelCase koru. Frontend bozulmaz	1,5 sa	1.2	B-16
1.5	`BaseModel`'den `name`'i çıkar; `id`, `created_at`, `updated_at` bırak. `Project`'e kendi `name` alanını ekle	45 dk	1.2	B-10, B-11
1.6	`get_by_slug`'ı `BaseRepository`'den çıkarıp `ProjectRepository(BaseRepository[Project])` içine taşı; `update`'i `exclude_unset=True` ile partial update destekler hale getir	1 sa	1.5	B-07, B-09
1.7	`PUT`/`PATCH`/`DELETE` endpoint'lerini ekle; `Page[T]` generic sayfalama şeması (`items`, `total`, `page`, `size`)	1,5 sa	1.6	B-07, B-21
1.8	`db_dependency`'ye `try/except/rollback` ekle	20 dk	—	B-08

Kontrol noktası: `/docs`'ta projects için 5 endpoint görünüyor; POST `id` istemiyor; PATCH tek alan güncelleyebiliyor.

Faz 2 — Test Altyapısı · Gün 3 (20 Ağu) · ~4 sa

Özelliklerden **önce** kuruluyor. 12 gün boyunca 4 modül ekleyeceksin; test yoksa her ekleme öncekini sessizce bozabilir.

#	Görev	Efor	Bağımlılık
2.1	`pytest`, `pytest-asyncio`, `httpx` ekle; `tests/conftest.py`: in-memory SQLite async engine, `AsyncClient` fixture, `app.dependency_overrides` ile session override	2 sa	Faz 1
2.2	`test_projects.py`: create / list / get / update / delete + 404 + validation (422) + sayfalama testleri	2 sa	2.1

Kontrol noktası: `pytest` yeşil, en az 10 test.

Faz 3 — Alembic ve PostgreSQL Geçişi · Gün 4 (21 Ağu, Cum) · ~5 sa

Postgres'e geçişi **erken** yapıyoruz. Sebep: SQLite'ta çalışan `JSON` kolonu Postgres'te farklı davranabilir; bunu deployment gününde keşfetmek istemezsin.

#	Görev	Efor	Bağımlılık
3.1	`alembic init -t async alembic`; `env.py`'yi SQLModel metadata ve `settings.DATABASE_URL` ile bağla	1,5 sa	Faz 1
3.2	İlk migration'ı üret; `init_db()` / `create_all` çağrısını `lifespan`'den kaldır	1 sa	3.1
3.3	`docker-compose.yml`'e `postgres:17-alpine` servisi ekle; `asyncpg` bağımlılığını ekle; `DATABASE_URL`'i env'den geçir	1,5 sa	3.2
3.4	Migration'ı ve tüm testleri Postgres'e karşı çalıştır; `JSON` → `JSONB` davranışını doğrula	1 sa	3.3

Kontrol noktası: Boş bir Postgres'te `alembic upgrade head` şemayı kuruyor; `pytest` hem SQLite hem Postgres'te yeşil.

Dikkat: Faz 1 bitmeden bu fazı başlatma — tüm şema değişiklikleri ilk migration'a girsin, sonra 6 tane düzeltme migration'ı yazmak zorunda kalma.

Faz 4 — Blog (Post) Modülü · Gün 5–6 (22–23 Ağu, hafta sonu) · ~8 sa

#	Görev	Efor	Bağımlılık
4.1	`Post` modeli: `title`, `slug` (unique+index), `summary`, `content` (`sa_column=Column(Text)`), `cover_image_url`, `tags` (JSON), `status` (draft/published enum), `published_at`, `reading_time`	2 sa	Faz 3
4.2	`PostCreate` / `PostUpdate` / `PostRead` şemaları; `python-slugify` ile Türkçe karakter → ASCII slug üretimi (ör. "Yazılım Mühendisliği" → `yazilim-muhendisligi`)	1,5 sa	4.1
4.3	`PostRepository`: slug ile getir, yalnızca published listele, tag'e göre filtrele, `published_at DESC` sırala	1,5 sa	4.2
4.4	`/api/v1/posts` endpoint'leri: public list (sadece published), public get-by-slug, admin CRUD	2 sa	4.3
4.5	Alembic migration + `test_posts.py` (draft'ın public listede görünmediğini de test et)	1 sa	4.4

Karar noktası: Markdown'ı backend'de mi HTML'e çevireceksin? **Öneri: hayır.** Ham markdown dön, frontend render etsin. Daha esnek olur ve XSS sorumluluğu tek yerde toplanır.

Faz 5 — Admin Auth (JWT) · Gün 7–8 (24–25 Ağu) · ~8 sa

#	Görev	Efor	Bağımlılık
5.1	`User` modeli: `email` (unique), `hashed_password`, `is_active`, `is_admin` + migration	1 sa	Faz 3
5.2	`core/security.py`: `pwdlib[argon2]` ile hash/verify, `PyJWT` ile access token üret/çöz	2 sa	5.1
5.3	`POST /api/v1/auth/login`: `OAuth2PasswordRequestForm` → access token; `GET /auth/me`	1,5 sa	5.2
5.4	`dependencies.py` → `get_current_admin`: `OAuth2PasswordBearer` + token doğrulama + 401/403	1,5 sa	5.3
5.5	Projects ve Posts'un tüm `POST`/`PUT`/`PATCH`/`DELETE` endpoint'lerini `Depends(get_current_admin)` ile koru; okuma public kalsın	1 sa	5.4
5.6	`scripts/create_admin.py`: env'den e-posta+parola alıp admin oluşturur. **Public register endpoint'i açma**	30 dk	5.2
5.7	`test_auth.py`: token'sız yazma → 401, geçersiz/süresi dolmuş token → 401, geçerli token → 200	1 sa	5.5

Kütüphane notu: FastAPI'nin resmî dokümanı artık `python-jose` yerine **PyJWT**, `passlib` yerine **pwdlib[argon2]** kullanıyor. `passlib` uzun süredir bakımsız — internetteki eski tutorial'lara göre değil, resmî dokümana göre ilerle.

Güvenlik notu: `JWT_SECRET`'ı `.env`'den al, asla repoya koyma. Refresh token bu kapsamda gereksiz — 60 dakikalık access token + yeniden login yeterli.

Faz 6 — Medya (ImageKit) ve İletişim Formu · Gün 9–10 (26–27 Ağu) · ~8,5 sa

#	Görev	Efor	Bağımlılık
6.1	`imagekitio`'yu **sürüm pinleyerek** ekle; `IMAGEKIT_PUBLIC_KEY` / `PRIVATE_KEY` / `URL_ENDPOINT` ayarlarını `Settings`'e ekle	30 dk	Faz 5
6.2	`services/media_service.py`: MIME ve boyut doğrulama (maks. 5 MB; jpeg/png/webp), ImageKit'e yükleme, dönen `url` + `file_id`'yi kaydetme	2,5 sa	6.1
6.3	`POST /api/v1/media/upload` (admin-only, `UploadFile`) ve `DELETE /api/v1/media/{file_id}`	1,5 sa	6.2
6.4	`ContactMessage` modeli (`name`, `email`, `subject`, `message`, `ip`, `is_read`) + migration	45 dk	Faz 3
6.5	`POST /api/v1/contact`: `EmailStr` doğrulaması, honeypot alanı, `slowapi` ile IP başına rate limit (ör. 3/saat)	1,5 sa	6.4
6.6	E-posta bildirimi: `fastapi-mail` veya Resend API ile kendine bildirim; `BackgroundTasks` ile gönder (isteği bloklamasın)	1,5 sa	6.5
6.7	`GET /api/v1/contact` — admin-only mesaj listesi	30 dk	6.5

Kritik SDK uyarısı: `imagekitio` Python SDK'sının güncel sürümü **5.8.0** (14 Temmuz 2026) ve API yüzeyi tamamen değişti: artık `ImageKit(private_key=...)` + `client.files.upload(file=..., file_name=...)` şeklinde, ayrıca async client'ı var. İnternetteki 3.x/4.x örnekleri **çalışmaz**. Resmî dokümana bak, sürümü `requirements.txt`'te pinle.

Mimari öneri (zaman kalırsa 6.2–6.3 yerine): Backend dosyayı hiç almasın; yalnızca **imzalı upload parametresi** üretsin (`GET /media/auth`), dosya tarayıcıdan doğrudan ImageKit'e gitsin. Droplet bandwidth'ini korur, private key hiç dışarı çıkmaz, ve büyük dosyalar API'yi meşgul etmez.

Kapsam esnekliği: Takvim sıkışırsa ilk kesilecek kalem **6.6 (e-posta bildirimi)**. Mesaj DB'ye yazılır, admin listesinden okunur; e-posta sonraki iterasyona kalır.

Faz 7 — Docker ve Droplet Deployment · Gün 11–13 (28–30 Ağu) · ~10,5 sa

#	Görev	Efor	Bağımlılık
7.1	Multi-stage `Dockerfile`: builder + `python:3.12-slim` runtime, non-root user, katman önbelleği için önce bağımlılıklar	2 sa	Faz 6
7.2	`docker-compose.yml`: `api` + `db` (postgres, named volume) + `caddy` reverse proxy	2 sa	7.1
7.3	Entrypoint script: konteyner başlarken `alembic upgrade head` çalıştır	1 sa	7.2
7.4	`GET /health` (DB ping dahil) + `core/logging.py`'de JSON structured logging + `middleware.py`'de request-id	1,5 sa	—
7.5	Droplet hazırlığı: Docker + Compose kurulumu, `ufw` (22/80/443), swap, non-root deploy kullanıcısı, SSH key auth	1,5 sa	—
7.6	Domain yönlendirme + Caddy ile otomatik Let's Encrypt HTTPS	1 sa	7.5
7.7	`CORS_ORIGINS`'i gerçek frontend domain'ine ayarla; production'da `/docs` ve `/redoc`'u kapat veya auth arkasına al	30 dk	7.6
7.8	Günlük `pg_dump` yedeği (cron, 7 gün saklama) ve geri yükleme prosedürünü bir kez dene	1 sa	7.6

Kritik: `.env` dosyasını droplet'e **git üzerinden değil**, `scp` ile veya elle taşı. Bir kez repoya girerse geçmişten temizlemek acı verici.

Faz 8 — Kapanış ve Devreye Alma · Gün 14 (31 Ağu, Pzt) · ~4 sa

#	Görev	Efor	Bağımlılık
8.1	GitHub Actions: her push'ta `pytest` + `ruff check`	1,5 sa	Faz 2
8.2	Frontend'i canlı API'ye bağla; uçtan uca smoke test (proje listesi, blog listesi, admin login, görsel yükleme, iletişim formu)	1 sa	Faz 7
8.3	README ve `.env.example`'ı son haline getir; `/docs`'taki endpoint açıklamalarını gözden geçir	1 sa	8.2
8.4	Başarı kriterleri listesini tek tek doğrula ve işaretle	30 dk	8.3

5. Zaman Çizelgesi

Gün	Tarih	Faz	Ana çıktı
1	18 Ağu, Sal	Faz 0	Proje temiz bir makinede kurulabiliyor
2	19 Ağu, Çar	Faz 1	Router aggregation + şema ayrımı
3	20 Ağu, Per	Faz 1 + 2	Project CRUD tamam, testler yeşil
4	21 Ağu, Cum	Faz 3	Alembic çalışıyor, Postgres ayakta
5	22 Ağu, Cmt	Faz 4	Post modeli + şemalar + repository
6	23 Ağu, Paz	Faz 4	Blog endpoint'leri + testler
7	24 Ağu, Pzt	Faz 5	User modeli + security katmanı
8	25 Ağu, Sal	Faz 5	Login + korumalı yazma endpoint'leri
9	26 Ağu, Çar	Faz 6	ImageKit görsel yükleme
10	27 Ağu, Per	Faz 6	İletişim formu + rate limit + e-posta
11	28 Ağu, Cum	Faz 7	Dockerfile + compose çalışıyor
12	29 Ağu, Cmt	Faz 7	Droplet hazır, HTTPS aktif
13	30 Ağu, Paz	Faz 7	Yedekleme + health + logging
14	31 Ağu, Pzt	Faz 8	CI, uçtan uca test, teslim

Tampon: Takvimde ayrı bir tampon günü yok — bu bilinçli. Yoğun tempoda tampon, kapsamdan çıkarılabilir kalemlerle sağlanır. Sıkışırsan sırayla şunları ertele: **6.6 (e-posta bildirimi)** → **8.1 (GitHub Actions)** → **7.8 (yedekleme cron'u, ama manuel bir pg_dump mutlaka al)**.

Not: 30 Ağustos Zafer Bayramı. O günü boş bırakmak istersen Faz 7'yi 11–12. günlere sıkıştır veya teslimi 1 Eylül'e kaydır.

6. Riskler ve Önlemler

Risk	Etki	Önlem
ImageKit SDK 5.x'in API'si eski tutorial'lardan farklı	Faz 6'da yarım gün kaybı	Sadece resmî dokümanı kullan, sürümü pinle, ilk iş küçük bir upload denemesi yap
SQLite → Postgres'te `JSON` kolon davranış farkı	Deployment gününde sürpriz	Faz 3'te (gün 4) erken geçiliyor, testler Postgres'e karşı da koşuyor
Yerel Python 3.14 ↔ Docker 3.12 farkı	"Bende çalışıyordu"	Docker Faz 3'te compose ile devreye giriyor; geliştirmenin son yarısı konteyner içinde
14 günde 4 özellik + deployment iddialı	Kapsam taşması	Öncelik sırası belli: auth > blog > deployment > medya > e-posta. Kesme sırası Bölüm 5'te
`.env` veya JWT secret'ın repoya sızması	Güvenlik	Faz 0.1'de `.gitignore` düzeltiliyor; `.env` droplet'e `scp` ile taşınıyor
Migration'ları Faz 1'den önce üretmek	6 tane gereksiz düzeltme migration'ı	Faz 3'ün bağımlılığı açıkça Faz 1
Korumasız API'nin canlıya çıkması	Portfolyona yabancı içerik	Faz 5, Faz 7'den (deployment) önce; sıra değiştirilmemeli

7. İlk Adım

Bugün (18 Ağustos) yapılacak ilk üç şey, sırasıyla:

9. `.gitignore`'u genişlet ve `myvenv/`, `*.db` dosyalarını git takibinden çıkar.
10. `requirements.txt`'teki `aiosqltite` yazım hatasını düzelt, `pydantic-settings` ve `greenlet` ekle.
11. Boş bir klasöre klonlayıp `pip install -r requirements.txt` + `uvicorn app.main:app` ile gerçekten kurulabildiğini doğrula.

Bu üçü bitmeden Faz 1'e geçme — sonraki her faz bunların üzerine kuruluyor.

Kaynaklar

- FastAPI — OAuth2 with Password (and hashing), Bearer with JWT tokens — https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/ — PyJWT ve `pwdlib[argon2]` önerisi
- imagekitio — PyPI — https://pypi.org/project/imagekitio/ — v5.8.0, 14 Temmuz 2026
- SlowApi Documentation — https://slowapi.readthedocs.io/ — rate limiting
- pwdlib — PyPI — https://pypi.org/project/pwdlib — modern parola hashleme
- The ultimate async setup: FastAPI, SQLModel, Alembic, Pytest — https://medium.com/@estretyakov/the-ultimate-async-setup-fastapi-sqlmodel-alembic-pytest-ae5cdcfed3d4
- FastAPI Alembic Migrations with PostgreSQL, SQLModel, and Docker — Keyhole Software — https://keyholesoftware.com/fastapi-alembic-migrations-with-postgresql-sqlmodel-and-docker/
- Docker in Practice #1: Containerizing FastAPI — uv, Multi-stage, non-root — https://schoolofweb.net/en/posts/docker-practice-1/